

AI GATEWAY

Multi-Tenant · Multi-Cloud · Data Ownership

Scaling Patterns · Isolation Architecture · Data Sovereignty · Cross-Cloud
Governance · Tenant Lifecycle Management

CHAPTER —

Table of Contents

A

The Multi-Tenancy Problem Space

- Tenancy Models & Trade-offs
- Isolation Failure Modes

B

Multi-Tenant Gateway Architecture

- Control Plane vs Data Plane Isolation
- Namespace, Shard & Cell Patterns
- Tenant Lifecycle Management

C

Multi-Cloud Gateway Design

- Cloud-agnostic Routing Fabric
- Cross-Cloud Failover & Latency Routing
- Provider Abstraction Layer

D

Data Ownership & Sovereignty

- Data Residency Enforcement
- Tenant Data Isolation Guarantees
- Prompt & Completion Data Lifecycle
- Right-to-Erasure (GDPR Art. 17)

E

Scaling Architecture

- Horizontal & Vertical Scaling Patterns
- Token-throughput Autoscaling
- Global Anycast & Edge PoPs

F

Cost Attribution & FinOps at Scale

- Chargeback Models
- Budget Guardrails per Tenant

G

Governance at Scale

- Policy Federation
- Audit at Multi-tenant Scale
- Compliance Matrix by Cloud & Region

H

Reference Patterns & Snippets

CHAPTER A

The Multi-Tenancy Problem Space

As enterprises scale AI capabilities across business units, geographies, products, and customer-facing SaaS offerings, the AI gateway must serve dozens to thousands of distinct **tenants** — each with unique identity, data, quota, policy, and compliance requirements. The naive approach of shared infrastructure without isolation boundaries creates catastrophic risks: data cross-contamination, quota exhaustion by noisy neighbours, compliance scope explosion, and blast-radius amplification on security incidents.

A.1 Tenancy Models & Trade-offs

Three primary tenancy models exist, positioned on a spectrum from maximum isolation (maximum cost) to maximum density (minimum cost). The choice is driven by regulatory classification of tenant data, SLA tier, and scale economics:

SILO	Dedicated gateway + infra per tenant	■ Maximum isolation ■ Zero noisy-neighbour ■ Simple audit scope	■ Highest cost ■ Slow provisioning ■ Operational complexity	<i>Regulated: HIPAA, FedRAMP High-value enterprise accounts</i>
BRIDGE	Shared gateway, isolated namespaces & dedicated DB schemas per tenant	■ Strong isolation ■ Moderate cost ■ Fast provisioning	■ Namespace escape risk ■ Shared control plane	<i>Mid-market SaaS Internal BU isolation</i>
POOL	Fully shared gateway, tenant ID as runtime context only	■ Lowest cost ■ Highest density ■ Simplest ops	■ Weakest isolation ■ Noisy neighbour risk ■ Widest blast radius	<i>Free/trial tiers Low-risk internal tools</i>

A.2 Isolation Failure Modes

Understanding failure modes is prerequisite to designing mitigations. The following taxonomy covers the critical ways multi-tenant AI gateway isolation breaks down in production:

Token Budget Cross-contamination

Problem: A tenant's token counter stored in a shared Redis key namespace collides with another tenant's key due to inadequate key prefixing. Tenant A exhausts Tenant B's daily budget or reads Tenant B's usage telemetry.

Mitigation: Mitigation: Tenant-scoped Redis keyspaces with ACL isolation per tenant. Key schema: `gw:{tenant_id}:quota:{model}:{window}`. Redis AUTH per tenant key range.

Prompt/Completion Data Leakage via Cache

Problem: A semantically similar prompt from Tenant A hits a vector cache entry populated by Tenant B's prior request. Tenant B's proprietary completion is returned to Tenant A.

Mitigation: Mitigation: Cache partition by `tenant_id` as mandatory primary key. Vector index is per-tenant (separate pgvector schema or Qdrant collection). Cross-tenant cache lookups are architecturally impossible.

Rate-limit Bypass via Header Spoofing

Problem: A malicious tenant crafts requests with a spoofed X-Tenant-ID header, routing their traffic against another tenant's quota bucket.

Mitigation: Mitigation: Tenant identity is derived exclusively from verified JWT claims (`sub`, `tenant_id`) or mTLS certificate SAN — never from user-supplied headers. Headers are stripped and re-injected by the gateway after authentication.

Audit Log Commingling

Problem: All tenant request logs written to a shared stream with insufficient partitioning. Compliance audit for Tenant A inadvertently surfaces Tenant B's prompt content.

Mitigation: Mitigation: Separate log streams per tenant (Kafka topic-per-tenant or S3 prefix-per-tenant with bucket policies). SIEM ingestion filters enforced at the collector level.

Model Version Drift

Problem: A gateway-wide model upgrade (e.g., GPT-4o → GPT-4o-mini for cost optimisation) changes output quality for tenants who pinned contract SLAs against specific model versions.

Mitigation: Mitigation: Per-tenant model pinning stored in tenant configuration registry. Upgrade rollouts use tenant-scoped canary deployments with A/B quality comparison.

CHAPTER B

Multi-Tenant Gateway Architecture

B.1 Control Plane vs Data Plane Isolation

The gateway separates its concerns into a **Control Plane** (configuration, policy, identity, quota) and a **Data Plane** (request processing, routing, provider communication). Isolation strategies differ by plane:

CONTROL	Tenant Registry ■ Policy Store (OPA) ■ Quota Engine ■ Config API ■ Admin Portal
SYNC	Config Push (gRPC) ■ Policy Distribution (bundle) ■ Quota Sync (Redis)
DATA	Auth Plugin ■ Rate Limiter ■ Cache (tenant-scoped) ■ Router ■ Provider Proxy
PROVIDERS	OpenAI ■ Anthropic ■ Azure OpenAI ■ Bedrock ■ Vertex AI ■ On-prem LLM
OBSERVE	Per-tenant Metrics ■ Isolated Audit Logs ■ Tenant Cost Dashboard

The control plane is a singleton per region, managing all tenant configurations. The data plane is horizontally scaled and stateless — tenant context is resolved per-request from JWT claims without shared memory. This allows the data plane to scale independently of control plane operations.

B.2 Namespace, Shard & Cell Patterns

Namespace Pattern

All gateway resources (routes, plugins, consumers, credentials) are namespaced under a `tenant_id` prefix. In Kong, this maps to separate Workspaces. In Kubernetes, to separate Namespaces with NetworkPolicies. Namespaces share the same gateway binary but enforce logical separation via RBAC and plugin scoping.

■ Namespace Pattern – snippet

```
# Kong Workspace per tenant (Admin API)
POST /workspaces { "name": "tenant-acme-corp" }
POST /tenant-acme-corp/consumers { "username": "acme-service-a" }
POST /tenant-acme-corp/plugins {
  "name": "ai-rate-limiting",
  "config": { "limit": [{"day": 1000000}] } # tenant-specific quota
}
```

Shard Pattern

High-throughput tenants are assigned dedicated gateway shard(s) — isolated Kubernetes Deployments with dedicated node pools. A global Layer-7 dispatcher routes tenant traffic to the correct shard based on `tenant_id` extracted from the JWT. Shards share no Redis, no vector cache, and no audit log infrastructure.

■ Shard Pattern – snippet

```
# Nginx upstream shard dispatcher (snippet)
```

```
map $jwt_tenant_id $backend_shard {
  ~^acme-.* shard-enterprise-a; # dedicated shard
  ~^startup-.* shard-pooled-b; # shared shard
  default shard-pooled-default;
}

upstream shard-enterprise-a {
  server gw-enterprise-a-1.internal:8000;
  server gw-enterprise-a-2.internal:8000;
}
```

Cell Architecture

For hyperscale (10,000+ tenants), the gateway adopts a Cell architecture: a cell is a fully self-contained stack (gateway + cache + quota store + audit log) serving a cohort of ~500 tenants. Cells are independently deployable, upgradable, and fault-isolated. A global cell-router maps tenant_id → cell_id via a consistent hash ring, allowing cell addition without rehashing all tenants.

■ Cell Architecture – snippet

```
# Cell assignment via consistent hash (Python concept)
ring = ConsistentHashRing(vnodes=150)
ring.add_nodes(['cell-us-east-1', 'cell-us-west-2', 'cell-eu-west-1'])
def route_tenant(tenant_id):
  cell = ring.get_node(tenant_id) # deterministic
  return CELL_ENDPOINTS[cell]
```

B.3 Tenant Lifecycle Management

Tenant onboarding and offboarding must be fully automated via a Tenant Management API. Manual provisioning at scale introduces drift, security gaps, and compliance violations:

Onboard	Tenant record created · Workspace/Namespace provisioned · Virtual keys issued · Quota bucket initialised · OPA policy bundle deployed · Audit log stream created · Tenant admin notified
Suspended	All virtual keys revoked · Rate limit set to 0 · Active connections drained gracefully · Data plane routes disabled · Control plane config frozen · Audit log sealed
Offboard	All keys deleted · Routes removed · Cache entries purged · Audit logs archived to cold storage with retention policy · Data deletion workflow triggered (GDPR Art. 17) · Compliance certificate of deletion issued

CHAPTER C

Multi-Cloud Gateway Design

Multi-cloud AI gateway deployments address three distinct organisational drivers: **resilience** (avoid single-cloud outage dependency), **compliance** (data residency requires specific cloud regions or on-prem), and **best-of-breed** (Azure OpenAI for Microsoft 365 integration, Bedrock for AWS-native workloads, Vertex for GCP-native ML pipelines). The gateway must present a unified plane while being physically deployed across AWS, Azure, GCP, and on-prem simultaneously.

C.1 Cloud-Agnostic Routing Fabric

GLOBAL LB	Cloudflare/Anycast DNS ■ GeoDNS / Latency routing ■ Health-based failover
REGIONAL GW	AWS: Kong on EKS (us-east-1) ■ Azure: Kong on AKS (eu-west) ■ GCP: Kong on GKE (asia-se)
MESH LINK	WireGuard tunnels / Transit Gateway ■ mTLS cross-cloud service accounts ■ BGP anycast (on-prem)
PROVIDERS	AWS Bedrock (same-cloud preference) ■ Azure OpenAI (same-cloud) ■ GCP Vertex (same-cloud) ■ OpenAI (any cloud)
CONTROL	Global Kong Admin API (read-only replicas) ■ Centralised OPA bundle server ■ Shared Vault (HA active-active)

The routing fabric follows a **same-cloud-preferred** policy: a request entering the AWS regional gateway prefers AWS Bedrock models to avoid cross-cloud data egress costs and latency. Cross-cloud routing is triggered only on health failure, capacity exhaustion, or explicit tenant policy override.

C.2 Cross-Cloud Failover & Latency Routing

■ Cross-cloud failover – Kong upstream config (snippet)

```
upstreams:
- name: llm-global
algorithm: latency # active latency measurement
healthchecks:
active:
http_path: /health
healthy: { interval: 10, successes: 2 }
unhealthy: { interval: 5, http_failures: 3 }
targets:
- target: gw-aws-us-east.internal:8000 weight: 100
- target: gw-azure-eu-west.internal:8000 weight: 80
- target: gw-gcp-asia-se.internal:8000 weight: 60
- target: gw-onprem-dc1.internal:8000 weight: 40
```

C.3 Provider Abstraction Layer

The Provider Abstraction Layer (PAL) normalises heterogeneous provider APIs into a single internal schema, allowing the routing layer and application layer to remain provider-agnostic. Schema translation is performed in the gateway plugin pipeline, not in application code:

Provider	Native API Schema	Auth Mechanism	Streaming Format	PAL Translation
OpenAI	/v1/chat/completions	Bearer API Key	SSE text/event-stream	Passthrough (reference)
Anthropic Claude	/v1/messages	x-api-key header	SSE application/json	messages→content mapping
AWS Bedrock	/model/{id}/invoke-stream	SigV4 signed request	chunked binary	InvokeModel→OpenAI schema
Azure OpenAI	/openai/deployments/{d}/chat	api-key + AAD OBO	SSE (same as OAI)	deployment ID mapping
Google Vertex	/v1/projects/{p}/generateContent	OAuth2 Bearer	SSE JSON	contents→messages mapping
Ollama (on-prem)	/api/chat	No auth (internal)	NDJSON stream	stream format convert

■ PAL – Bedrock to OpenAI schema translation (snippet)

```
-- Kong plugin: bedrock-normalizer
local function transform_request(body)
return {
  modelId = BEDROCK_MODEL_MAP[body.model],
  messages = map_openai_to_bedrock_messages(body.messages),
  inferenceConfig = {
    maxTokens = body.max_tokens,
    temperature = body.temperature,
  }
}
end

local function transform_response(bedrock_resp)
return { -- OpenAI shape
  choices = {{ message = {
    role = 'assistant',
    content = bedrock_resp.output.message.content[1].text
  }}}
}
end
```


CHAPTER D

Data Ownership & Sovereignty

Data ownership in multi-tenant AI gateways spans four distinct data categories, each requiring separate governance: **prompts** (input data, often containing PII or proprietary business context), **completions** (output data, potentially containing model-synthesised sensitive information), **metadata** (token counts, latency, consumer identity — operational telemetry), and **embeddings** (vector representations of tenant content stored in semantic caches). Each category has distinct ownership, residency, retention, and erasure requirements.

D.1 Data Residency Enforcement

Data residency enforcement ensures that a tenant's prompt and completion data never leaves a specified geographic jurisdiction. The gateway enforces residency at the routing layer, not as a post-hoc control:

Residency-Aware Routing

Each tenant profile carries a **data_residency** attribute (e.g., EU, US, APAC, IN). The router enforces a strict allow-list of provider endpoints that satisfy the residency requirement. Requests from EU-residency tenants can only be routed to Azure OpenAI Sweden Central, OpenAI EU endpoints, or AWS Bedrock eu-west-1 — regardless of latency or cost optimisation signals.

Egress Enforcement via Network Policy

Kubernetes NetworkPolicies and Istio AuthorizationPolicies at the egress gateway enforce that traffic from EU-namespace pods cannot reach US provider endpoints at the network layer — providing a defence-in-depth guarantee independent of the application-layer routing decision.

On-Premises Residency

For data classified as unable to leave corporate infrastructure (sovereign cloud, classified government workloads), the gateway routes exclusively to on-premises model serving (vLLM, Ollama, Triton) or private Azure Government / AWS GovCloud endpoints. The PAL ensures the same API surface regardless of where the model runs.

■ Residency-aware routing policy – OPA (snippet)

```
package ai.gateway.residency
# Allowed providers per residency zone
residency_allow = {
  "EU": {"azure-sweden", "aws-bedrock-eu-west-1", "onprem-eu-dc1"},
  "US": {"openai-us", "azure-east-us", "aws-bedrock-us-east-1"},
  "IN": {"azure-india-central", "aws-bedrock-ap-south-1"},
}
allow_route if {
  tenant := data.tenants[input.tenant_id]
  allowed := residency_allow[tenant.data_residency]
  input.target_provider in allowed
}
```

D.2 Tenant Data Isolation Guarantees

The following table defines the isolation guarantee level for each data type across the three tenancy models:

Data Type	SILO	BRIDGE	POOL
Prompts (in-flight)	Dedicated TLS endpoint	Shared endpoint, tenant JWT	Shared endpoint, tenant JWT
Completions	Dedicated TLS endpoint	Shared endpoint, filtered	Shared endpoint, filtered
Semantic Cache	Dedicated vector DB	Isolated collection/schema	Partitioned by tenant_id key
Quota counters	Dedicated Redis instance	Tenant-prefixed Redis keys	Tenant-prefixed Redis keys
Audit Logs	Dedicated S3 bucket	Tenant-prefixed S3 prefix	Tenant-tagged shared stream
Embeddings	Dedicated index	Isolated namespace/schema	Tenant_id partition key
Model config	Dedicated CRDs	Tenant workspace config	Tenant config object

D.3 Prompt & Completion Data Lifecycle

The gateway defines explicit data lifecycle policies for prompt and completion content, distinguishing between operational logging (for debugging and cost tracking) and compliance audit logging (for regulatory evidence). By default, **prompt and completion content is never stored** — only metadata is logged. Content logging is an explicit opt-in per tenant with additional encryption and access controls:

Transit	In-flight only. TLS 1.3 with perfect-forward-secrecy. No buffering except for streaming reassembly (max 30s window). Memory is zeroed after response dispatch.
Operational Log	Metadata only by default: timestamp, tenant_id, model, token_counts, latency, cache_hit, cost_usd. Content stored only if tenant opts in + DPO approval + field-level encryption with tenant-held key.
Compliance Audit	Immutable record: consumer_id, request_hash (SHA-256 of prompt), response_hash, model_version, provider, timestamp. Hashes enable audit without storing raw content. Retained per regulatory requirement (7yr HIPAA, 5yr SOC2).
Cache TTL	Vector cache entries carry configurable TTL (default 1 hour). Tenant can set TTL=0 to disable caching for sensitive workloads. Cache entries are encrypted with tenant-specific AES-256 key stored in Vault.
Erasure	Right-to-erasure pipeline: cache purge → operational log field masking → audit hash retention (hash retained, no raw content to erase) → certificate of deletion. SLA: 72 hours for GDPR, 30 days for CCPA.

D.4 Right-to-Erasure (GDPR Article 17)

■ Tenant data deletion workflow – API + operator script (snippet)

```
# Step 1: Trigger deletion via Tenant Management API
DELETE /v1/tenants/{tenant_id}/data
{ "scope": ["cache", "logs", "embeddings"], "reason": "GDPR_ART17" }

# Step 2: Gateway operator reconciler executes:
# -- Purge Redis keyspace: DEL gw:{tenant_id}:*
# -- Drop pgvector schema: DROP SCHEMA tenant_{id} CASCADE
```

```
# -- Mask operational logs: UPDATE logs SET prompt=NULL WHERE tenant=?  
# -- Archive audit hashes to cold store (hashes kept, content null)  
# -- Generate signed deletion certificate (timestamped, DPO countersigned)  
# Step 3: Verification  
GET /v1/tenants/{tenant_id}/data/audit  
# Returns: { status: DELETED, certificate_id: ..., completed_at: ... }
```

CHAPTER E

Scaling Architecture

E.1 Horizontal & Vertical Scaling Patterns

The gateway data plane is designed for horizontal scaling — stateless request processing with all state externalised to Redis (quota counters, session data) and the vector cache. Vertical scaling (larger instance types) is only applied to the semantic embedding computation path, which is CPU/GPU-intensive.

Stateless Data Plane Pods

Gateway instances hold zero in-memory tenant state. All quota, session, and cache state is in Redis Cluster. Any pod can serve any tenant. HPA scales on token-throughput (custom metric) and CPU. Target: 3–50 replicas per cell.

■ Config

```
hpa_metric: gateway_tokens_per_second hpa_target: 50,000 TPS per pod
```

Redis Cluster for Quota State

Redis Cluster (6 shards, 3 replicas each) handles quota counters using INCRBY with TTL. Consistent hashing distributes tenant keys across shards. Redis 7.x ACL isolates per-tenant keyspaces. CRDT-based counters handle cross-region quota synchronisation with bounded staleness (5s).

■ Config

```
redis_topology: 6-shard cluster replication: 3 replicas/shard consistency: eventual (5s)
```

Vector Cache Scaling

pgvector deployed as read-replica cluster: 1 primary (writes) + N read replicas (similarity lookups). Read traffic load-balanced by gateway plugin. At >1M cached embeddings, Qdrant distributed cluster is preferred for horizontal vector index scaling with HNSW indexing.

■ Config

```
cache_backend: pgvector (< 1M) → Qdrant cluster (> 1M) index: HNSW, ef=128
```

Control Plane Scaling

Kong Admin API with read-only replica nodes. Configuration changes hit the primary; data plane nodes pull configuration via Kong's declarative config sync (deck sync) every 30 seconds. At 10,000+ tenants, a dedicated Config Delivery Network (CDN-style) distributes tenant config bundles to regional data planes.

■ Config

```
sync_interval: 30s config_delivery: CDN-backed bundle server
```

E.2 Global Anycast & Edge PoPs

For globally distributed SaaS tenants, the gateway is deployed at edge Points of Presence (PoPs) using Cloudflare Workers, Fastly Compute@Edge, or AWS CloudFront Functions as a lightweight first-hop. The edge PoP handles:

- JWT validation and tenant identification (eliminates RTT to origin for auth failures)
- Rate-limit pre-check against edge-local counters (eventual consistency with origin quota)
- Semantic cache lookup against a CDN-proxied vector cache (read-only, refreshed every 60s)
- Request routing to the nearest regional gateway cluster based on tenant residency policy
- TLS termination and DDoS absorption (absorbs L3/L4 attack traffic before it reaches gateway fleet)

■ Cloudflare Worker – edge auth + rate pre-check (snippet)

```
export default {
  async fetch(request, env) {
    // 1. Validate JWT at edge (no origin round-trip)
    const jwt = await verifyJWT(request.headers.get('Authorization'), env.JWKS_URL)
    if (!jwt.valid) return new Response('Unauthorized', { status: 401 })
    // 2. Edge rate-limit pre-check (KV store, eventual consistency)
    const key = `rl:${jwt.tenant_id}:${getMinuteBucket()}`
    const count = await env.KV.get(key) ?? 0
    if (count > EDGE_LIMIT) return new Response('Too Many Requests', { status: 429 })
    await env.KV.put(key, count+1, { expirationTtl: 120 })
    // 3. Route to tenant's regional gateway
    const region = RESIDENCY_REGION[jwt.data_residency]
    return fetch(`https://${region}-gateway.corp/${request.url}`, request)
  }
}
```

CHAPTER F

Cost Attribution & FinOps at Scale

F.1 Chargeback Models

At multi-tenant scale, cost attribution must be granular enough to drive engineering behaviour (prompt optimisation, cache leverage) and support accurate financial chargeback to business units or external customers:

Token-based Chargeback

Gateway emits per-request telemetry: `prompt_tokens`, `completion_tokens`, `model`, `provider`, `tenant_id`, `team_id`, `feature_id`. FinOps pipeline (OpenCost + custom Prometheus rules) aggregates `cost = tokens × model_rate_per_token`. Grafana dashboards show cost by tenant / team / model / feature with drill-down.

■ Formula

```
cost_formula: (prompt_tokens * input_rate) + (completion_tokens * output_rate)
```

Request-based Chargeback

For tenants on flat-rate contracts, cost is allocated per request regardless of token count. Gateway tags each request with a `request_unit` (RU) weight by model tier. GPT-4o = 10 RU, GPT-4o-mini = 1 RU, Claude Haiku = 1.5 RU. Monthly RU consumption drives invoice generation.

■ Formula

```
cost_formula: requests * model_request_unit_weight
```

Cache Savings Credit

Tenants who generate cache hits effectively subsidise their peers. FinOps model credits the cache-hitting tenant: if a cache hit returns a \$0.05 completion at zero provider cost, the tenant is charged a `cache_fee` (\$0.001) rather than full provider cost. Cache hit rate is a first-class metric in tenant cost dashboards.

■ Formula

```
charge: min(provider_cost, cache_fee) # cache_fee << provider_cost
```

F.2 Budget Guardrails per Tenant

■ Budget exhaustion policy – Kong plugin config (snippet)

```
plugins:
- name: ai-quota-guardian
config:
tenant_id: '{{ jwt.tenant_id }}'
budgets:
- window: day
limit_usd: 500.00
```

on_exceed: throttle # 429 with Retry-After
warn_at_pct: 80 # alert at \$400
- window: month
limit_usd: 10000.00
on_exceed: queue # queue in Redis, process when budget resets
escalate_to: finops@corp.com
model_overrides:
gpt-4o:
on_exceed: downgrade_to # auto-downgrade to gpt-4o-mini
downgrade_model: gpt-4o-mini

CHAPTER G

Governance at Scale

G.1 Policy Federation

At multi-tenant scale, a monolithic policy file is unmanageable. The gateway adopts a federated policy architecture using OPA's bundle mechanism: global base policies are authored by the Platform team; tenant-specific overrides are authored by Tenant Admins within a constrained sandbox, preventing privilege escalation.

■ OPA Bundle hierarchy (snippet)

```
# Bundle server serves layered bundles:
# /bundles/global/base.tar.gz – platform-authored, enforced everywhere
# /bundles/tenant/{id}/policy.tar.gz – tenant-authored, sandboxed scope
# Global base policy (enforced, cannot be overridden):
package ai.gateway.global
deny if { input.model in BANNED_MODELS_GLOBAL } # e.g. ['dall-e-2']
deny if { input.prompt_injection_score > 0.9 }
# Tenant override (tenant can restrict further, not expand):
package ai.gateway.tenant
allow_model if {
  not ai.gateway.global.deny # global deny takes precedence
  input.model in data.tenant_allowed_models[input.tenant_id]
}
```

G.2 Audit at Multi-Tenant Scale

Generating, storing, and querying audit logs for thousands of tenants at millions of requests per day requires a purpose-built audit architecture:

- Kafka topic-per-tenant for real-time audit event streaming; compacted topics for latest-state queries
- S3 with tenant-prefixed partition keys; S3 Object Lock (WORM) for tamper-proof retention
- Athena/BigQuery partitioned queries allow tenant-scoped audit export without cross-tenant data access
- Audit log schema is immutable once deployed; schema evolution only via additive fields (never delete/rename)
- Tenant self-service audit portal: tenants query their own audit log via scoped API credentials — no platform team involvement required
- Cross-tenant audit aggregation (for platform-level security analytics) uses separate anonymised data pipeline with PII removed

G.3 Compliance Matrix by Cloud & Region

Regulation	Scope	AWS (us-east)	Azure (eu-west)	GCP (asia-se)	On-prem
GDPR	EU persons' data	■ Disallowed	■ Allowed	■ Disallowed	■ Allowed

Regulation	Scope	AWS (us-east)	Azure (eu-west)	GCP (asia-se)	On-prem
HIPAA	US health data	■ BAA signed	■ BAA signed	■ BAA signed	■ Allowed
SOC 2 Type II	Service trust	■ Covered	■ Covered	■ Covered	■■ Self-cert
DPDP (India)	IN persons' data	■ Disallowed	■ Disallowed	■■ Mumbai PoP	■ Preferred
FedRAMP High	US Gov data	■ GovCloud	■ Gov Cloud	■ Disallowed	■ SILO only
EU AI Act	High-risk AI	■■ EU mirror	■ Native EU	■	■ Full ctrl
PCI DSS	Cardholder data	■ + no log	■ + no log	■ + no log	■ + no log

CHAPTER H

Reference Patterns & Snippets

H.1 Multi-Tenant Gateway — Decision Checklist

Tenancy Model	Silo / Bridge / Pool decided per tenant tier based on regulatory classification
Identity	Tenant_id sourced from verified JWT claim only — never HTTP header
Cache	Vector cache partitioned per tenant_id as primary key — cross-tenant lookups impossible
Quota	Redis keys prefixed gw:{tenant_id}: — ACL-enforced keyspace isolation
Residency	OPA policy enforces provider allow-list per tenant data_residency attribute
Audit Logs	Separate Kafka topic or S3 prefix per tenant — no shared stream
Data Deletion	Automated erasure pipeline with 72h SLA and signed deletion certificate
Cost	Per-request token telemetry → FinOps pipeline → chargeback dashboard
Policy	OPA bundle federation: global base (overrides all) + tenant sandbox (additive only)
Scaling	Stateless data plane pods + external Redis + vector cache → HPA on token throughput
Multi-cloud	Same-cloud-preferred routing + cross-cloud failover + Provider Abstraction Layer
Model Pinning	Per-tenant model version locked in control plane — upgrade via tenant-scoped canary

H.2 Capacity Sizing Reference

Scale Tier	Tenants	Peak RPS	Gateway Pods	Redis	Vector Cache	Audit Storage
Small	1–50	500	3 pods (2 CPU)	Single node 8GB	pgvector 1 node	100GB/month S3
Medium	50–500	5,000	10 pods (4 CPU)	Cluster 3-shard	pgvector 3 node	1TB/month S3
Large	500–5k	50,000	30 pods (8 CPU)	Cluster 6-shard	Qdrant 3-node	10TB/month S3
Hyperscale	5k–50k	500,000	Cell arch (3 cells 50 pods each)	Redis Enterprise	Qdrant 9-node cluster	100TB/month + Glacier

H.3 Data Ownership RACI

Activity	Platform Team	Tenant Admin	Data Owner	DPO / Legal
Define residency policy	Consult	Inform	Accountable	Responsible

Activity	Platform Team	Tenant Admin	Data Owner	DPO / Legal
Configure tenant residency	Responsible	Accountable	Inform	Consult
Enable content logging	Implement	Request	Accountable	Approve
Trigger data erasure	Execute	Accountable	Initiate	Audit
Audit log access (own data)	Provide access	Responsible	Review	Oversee
Cross-tenant security audit	Responsible	Informed	Informed	Accountable
Model version upgrade	Responsible	Approve/Veto	Inform	Consult
Budget limit configuration	Implement	Accountable	Inform	Consult

This addendum complements the Enterprise AI Gateway primary document. Multi-tenancy and multi-cloud topology choices are architecture decisions with long-term cost and compliance implications — review with your Data Protection Officer, Cloud Architect, and FinOps lead before committing to a tenancy model.